# Apresentação BD II

Evandro, Joel e Lucas

INSTITUTO FEDERAL
Espírito Santo

## Questão 1 - Só consultas REALIZADAS devem ser faturadas (regra de negócio — pode ser via trigger).

```
create or replace function verificar_consulta_realizada()
returns trigger as $$
    begin
        if not exists(
            select 1 from consulta
            where id_consulta = NEW.id_consulta
            and status = 'REALIZADA'
        ) then
            raise exception 'A consulta não pode ser faturda';
        end if;

        return new;
    end
    $$ language plpgsql


create trigger trg_verificar_consulta_realizada
before insert on fatura_item
for each row
execute function verificar_consulta_realizada();
```

INSTITUTO FEDERAL
Espírito Santo

## Questão 1 - Só consultas REALIZADAS devem ser faturadas (regra de negócio — pode ser via trigger).

Query | Query History

```
INSERT INTO fatura_item (id_fatura, id_consulta, valor)
VALUES (1, 1, 100.00);
```

Data Output | Messages | Notifications

```
ERROR:  A consulta não pode ser faturda
CONTEXT:  função PL/pgSQL verificar_consulta_realizada() linha 8 em RAISE

ERRO:  A consulta não pode ser faturda
SQL state: P0001
```

Query | Query History

```
INSERT INTO fatura_item (id_fatura, id_consulta, valor)
VALUES (1, 1, 100.00);

update consulta set status = 'REALIZADA'
where id_consulta = 1
```

Data Output | Messages | Notifications

```
INSERT 0 1

Query returned successfully in 61 msec.
```

INSTITUTO FEDERAL
Espírito Santo

## Questão 2 - Elabore uma Função SQL para exibir consultas realizadas por Convênio (Plano de Saúde).

```
CREATE OR REPLACE FUNCTION consultas_por_convenio(p_id_convenio INT)
RETURNS TABLE(
    paciente VARCHAR,
    medico VARCHAR,
    data TIMESTAMP,
    status VARCHAR
) AS $$
SELECT p.nome, m.nome, c.data_hora, c.status
FROM consulta c
JOIN paciente p ON p.id_paciente = c.id_paciente
JOIN medico m ON m.id_medico = c.id_medico
JOIN paciente_convenio pc ON pc.id_paciente = p.id_paciente
WHERE pc.id_convenio = p_id_convenio
AND c.status = 'REALIZADA';
$$ LANGUAGE sql;
```

INSTITUTO FEDERAL
Espírito Santo

## Questão 2 - Elabore uma Função SQL para exibir consultas realizadas por Convênio (Plano de Saúde).

- Todas as consultas estão com o status = ‘AGENDADA’, portanto:

```sql
UPDATE consulta
SET status = 'REALIZADA'
WHERE id_consulta IN (1,2);
```

```sql
select * from consultas_por_convenio(1)
```

```sql
select * from convenio where id_convenio = 1
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 — Page No: 1 of 1

| | paciente character varying | medico character varying | data timestamp wi |
|---|---|---|---|
| 1 | João Silva | Dr. Pedro Santos | 2026-03-25 0 |
| 2 | Maria Oliveira | Dra. Ana Costa | 2026-03-25 0 |

Showing rows: 1 to 1 — Page No: 1 of 1

| | id_convenio [PK] integer | nome character varying (100) | cnpj character va |
|---|---|---|---|
| 1 | 1 | Unimed | 00.000.000/ |

INSTITUTO FEDERAL
Espírito Santo

Questão 2 - Elabore uma Função SQL para exibir consultas realizadas por Convênio (Plano de Saúde).

Prova real:

```sql
SELECT
    p.nome AS paciente,
    m.nome AS medico,
    c.data_hora AS data,
    c.status,
    pc.id_convenio
FROM consulta c
JOIN paciente p
    ON p.id_paciente = c.id_paciente
JOIN medico m
    ON m.id_medico = c.id_medico
JOIN paciente_convenio pc
    ON pc.id_paciente = p.id_paciente
WHERE pc.id_convenio = 1
AND c.status = 'REALIZADA';
```

Showing rows: 1 to 2 Page No: 1 of 1

| | paciente character varying (100) | medico character varying (100) | data timestamp without time zone | status character varying (20) | id_convenio integer |
|---|---|---|---|---|---|
| 1 | Maria Oliveira | Dra. Ana Costa | 2026-03-25 09:00:00 | REALIZADA | 1 |
| 2 | João Silva | Dr. Pedro Santos | 2026-03-25 08:00:00 | REALIZADA | 1 |

INSTITUTO FEDERAL
Espírito Santo

## Questão 3 - Faça uma Função PLPGSQL para exibir consultas a serem realizadas em uma data específica.

```
create or replace function consultas_por_data(p_data DATE)
return table(
   paciente VARCHAR,
   medico VARCHAR,
   data TIMESTAMP
) as $$
begin
   return query
   select p.nome, m.nome, c.data_hora
   from consulta c
   join paciente p on p.id_paciente = c.id_paciente
   join medico m on m.id_medico = c.id_medico
   where DATE(c.data_hora) = p_data;
end;
$$ LANGUAGE plpgsql;


select * from consultas_por_data('2026-03-25');
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 — Page No: 1 of 1

| | paciente character varying | medico character varying | data timestamp w |
|---|---|---|---|
| 1 | Carlos Souza | Dr. Pedro Santos | 2026-03-25 1 |
| 2 | João Silva | Dr. Pedro Santos | 2026-03-25 0 |
| 3 | Maria Oliveira | Dra. Ana Costa | 2026-03-25 0 |

INSTITUTO FEDERAL
Espírito Santo

# [4] Implemente um Stored Procedure para a marcação de uma consulta em horário válido. Não esqueça de validar todos os dados de entrada.

```
CREATE OR REPLACE PROCEDURE sp_marcar_consulta(
  p_id_paciente INT,
  p_id_medico INT,
  p_data_hora TIMESTAMP,
  p_observacao TEXT DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
BEGIN
  IF p_id_paciente IS NULL OR p_id_medico IS NULL OR p_data_hora IS NULL THEN
    RAISE EXCEPTION 'id_paciente, id_medico e data_hora são obrigatórios.';
  END IF;

  IF NOT EXISTS (SELECT 1 FROM paciente WHERE id_paciente = p_id_paciente) THEN
    RAISE EXCEPTION 'Paciente % não existe.', p_id_paciente;
  END IF;

  IF NOT EXISTS (SELECT 1 FROM medico WHERE id_medico = p_id_medico) THEN
    RAISE EXCEPTION 'Médico % não existe.', p_id_medico;
  END IF;

  IF p_data_hora <= NOW() THEN
    RAISE EXCEPTION 'A consulta deve ser marcada para data/hora futura.';
  END IF;

  IF EXTRACT(ISODOW FROM p_data_hora) IN (6, 7) THEN
    RAISE EXCEPTION 'Consultas só podem ser marcadas de segunda a sexta-feira.';
  END IF;

  IF p_data_hora::TIME < TIME '07:00' OR p_data_hora::TIME >= TIME '19:00' THEN
    RAISE EXCEPTION 'Horário inválido. Faixa permitida: 07:00 até 18:59.';
  END IF;

  IF EXTRACT(MINUTE FROM p_data_hora) NOT IN (0, 30) THEN
    RAISE EXCEPTION 'Horário inválido. Somente horários em ponto ou meia-hora (:00 ou :30).';
  END IF;

  IF EXISTS (
    SELECT 1
    FROM consulta
    WHERE id_medico = p_id_medico
      AND data_hora = p_data_hora
      AND status = 'AGENDADA'
  ) THEN
    RAISE EXCEPTION 'Médico % já possui consulta agendada em %.', p_id_medico, p_data_hora;
  END IF;

  INSERT INTO consulta (id_paciente, id_medico, data_hora, status, observacao)
  VALUES (p_id_paciente, p_id_medico, p_data_hora, 'AGENDADA', p_observacao);
END;
$$;
```

INSTITUTO FEDERAL
Espírito Santo

# [5] Crie livremente duas visões (views) - uma materializada e outra não.

```sql
DROP VIEW IF EXISTS vw_agenda_consultas_dia;
CREATE VIEW vw_agenda_consultas_dia AS
SELECT
  c.id_consulta,
  c.data_hora::DATE AS data,
  TO_CHAR(c.data_hora, 'HH24:MI') AS horario,
  m.nome AS medico,
  p.nome AS paciente,
  c.status
FROM consulta c
JOIN medico m ON m.id_medico = c.id_medico
JOIN paciente p ON p.id_paciente = c.id_paciente;
```

```sql
DROP MATERIALIZED VIEW IF EXISTS mv_faturamento_convenio_mensal;
CREATE MATERIALIZED VIEW mv_faturamento_convenio_mensal AS
SELECT
  cv.id_convenio,
  cv.nome AS convenio,
  DATE_TRUNC('month', f.data_emissao)::DATE AS competencia,
  COUNT(DISTINCT fi.id_consulta) AS qtd_consultas_faturadas,
  COALESCE(SUM(fi.valor), 0)::NUMERIC(10,2) AS total_faturado
FROM convenio cv
LEFT JOIN fatura f ON f.id_convenio = cv.id_convenio
LEFT JOIN fatura_item fi ON fi.id_fatura = f.id_fatura
GROUP BY cv.id_convenio, cv.nome, DATE_TRUNC('month', f.data_emissao)
WITH DATA;
```

INSTITUTO FEDERAL
Espírito Santo

# [5] Crie livremente duas visões (views) - uma materializada e outra não.

```
-- [5] View comum.
SELECT * FROM vw_agenda_consultas_dia ORDER BY data, horario;

-- [5] View materializada.
REFRESH MATERIALIZED VIEW mv_faturamento_convenio_mensal;
SELECT * FROM mv_faturamento_convenio_mensal ORDER BY convenio, competencia;
```

```
 id_consulta |    data    | horario |      medico      |    paciente    |  status
-------------+------------+---------+------------------+----------------+-----------
           1 | 2026-03-25 | 08:00   | Dr. Pedro Santos | João Silva     | REALIZADA
           2 | 2026-03-25 | 09:00   | Dra. Ana Costa   | Maria Oliveira | REALIZADA
           3 | 2026-03-25 | 10:00   | Dr. Pedro Santos | Carlos Souza   | AGENDADA
           4 | 2026-12-01 | 10:30   | Dra. Ana Costa   | João Silva     | AGENDADA
(4 rows)

REFRESH MATERIALIZED VIEW
 id_convenio |     convenio      | competencia | qtd_consultas_faturadas | total_faturado
-------------+-------------------+-------------+-------------------------+----------------
           2 | Amil              | 2026-04-01  |                       0 |           0.00
           3 | Convenio MV Teste | 2026-04-01  |                       0 |           0.00
           1 | Unimed            | 2026-04-01  |                       2 |         480.00
(3 rows)
```

INSTITUTO FEDERAL
Espírito Santo

# [5] Crie livremente duas visões (views) - uma materializada e outra não.

```
\connect clinica

-- 1) Estado inicial da materialized view para o convênio de teste.
SELECT *
FROM mv_faturamento_convenio_mensal
where convenio = 'Convenio MV Teste'
ORDER BY convenio, competencia;

-- 2) Insere um novo convênio na tabela base (dado novo).
INSERT INTO convenio (nome, cnpj, telefone)
VALUES ('Convenio MV Teste', '99.999.999/0001-99', '(27) 99999-9999')
ON CONFLICT (cnpj) DO NOTHING;

-- Confirma que o dado foi inserido na tabela base.
SELECT id_convenio, nome, cnpj
FROM convenio
WHERE cnpj = '99.999.999/0001-99';
```

```
 id_convenio | convenio | competencia | qtd_consultas_faturadas | total_faturado
-------------+----------+-------------+-------------------------+----------------
(0 rows)

INSERT 0 1
 id_convenio |       nome        |        cnpj
-------------+-------------------+--------------------
           3 | Convenio MV Teste | 99.999.999/0001-99
(1 row)

 id_convenio | convenio | competencia | qtd_consultas_faturadas | total_faturado
-------------+----------+-------------+-------------------------+----------------
(0 rows)
```

```
-- 3) Consulta a materialized view SEM refresh: continua sem refletir a inserção.
SELECT *
FROM mv_faturamento_convenio_mensal
where convenio = 'Convenio MV Teste'
ORDER BY convenio, competencia;

-- 4) Atualiza a materialized view.
REFRESH MATERIALIZED VIEW mv_faturamento_convenio_mensal;

-- 5) Agora a materialized view passa a refletir o dado novo.
SELECT *
FROM mv_faturamento_convenio_mensal
where convenio = 'Convenio MV Teste'
ORDER BY convenio, competencia;
```

```
REFRESH MATERIALIZED VIEW
 id_convenio |     convenio      | competencia | qtd_consultas_faturadas | total_faturado
-------------+-------------------+-------------+-------------------------+----------------
           3 | Convenio MV Teste |             |                       0 |           0.00
(1 row)
```

INSTITUTO FEDERAL
Espírito Santo

[6] Estabeleça uma política de segurança por meio de um script que assegure a existência de, pelo menos, três grupos de usuários com cinco usuários a eles pertencentes. Defina o nível de privilégios destes grupos de usuários sobre os objetos do banco de dados.

```sql
-- Grupos (roles sem login)
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'grp_admin_clinica') THEN
    CREATE ROLE grp_admin_clinica NOLOGIN;
  END IF;

  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'grp_atendimento') THEN
    CREATE ROLE grp_atendimento NOLOGIN;
  END IF;

  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'grp_faturamento') THEN
    CREATE ROLE grp_faturamento NOLOGIN;
  END IF;
END;
$$;

-- Usuários (5 usuários)
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'usr_admin') THEN
    CREATE ROLE usr_admin LOGIN PASSWORD 'admin123';
  END IF;

  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'usr_recepcao1') THEN
    CREATE ROLE usr_recepcao1 LOGIN PASSWORD 'recepcao123';
  END IF;

  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'usr_recepcao2') THEN
    CREATE ROLE usr_recepcao2 LOGIN PASSWORD 'recepcao123';
  END IF;

  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'usr_faturista1') THEN
    CREATE ROLE usr_faturista1 LOGIN PASSWORD 'fatura123';
  END IF;

  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'usr_faturista2') THEN
    CREATE ROLE usr_faturista2 LOGIN PASSWORD 'fatura123';
  END IF;
END;
$$;
-- Associação dos usuários aos grupos
GRANT grp_admin_clinica TO usr_admin;
GRANT grp_atendimento TO usr_recepcao1, usr_recepcao2;
GRANT grp_faturamento TO usr_faturista1, usr_faturista2;
```

INSTITUTO FEDERAL
Espírito Santo

```sql
-- Endurecimento de segurança básico
REVOKE ALL ON SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL SEQUENCES IN SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL FUNCTIONS IN SCHEMA public FROM PUBLIC;

-- Permissões do grupo admin (acesso total no schema public)
GRANT USAGE, CREATE ON SCHEMA public TO grp_admin_clinica;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO grp_admin_clinica;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO grp_admin_clinica;
GRANT ALL PRIVILEGES ON ALL FUNCTIONS IN SCHEMA public TO grp_admin_clinica;

-- Permissões do atendimento
GRANT USAGE ON SCHEMA public TO grp_atendimento;
GRANT SELECT, INSERT, UPDATE ON paciente TO grp_atendimento;
GRANT SELECT ON medico, convenio, procedimento TO grp_atendimento;
GRANT SELECT, INSERT, UPDATE ON paciente_convenio TO grp_atendimento;
GRANT SELECT, INSERT, UPDATE ON consulta TO grp_atendimento;
GRANT SELECT ON vw_agenda_consultas_dia TO grp_atendimento;
GRANT USAGE, SELECT ON SEQUENCE paciente_id_paciente_seq TO grp_atendimento;
GRANT USAGE, SELECT ON SEQUENCE paciente_convenio_id_paciente_convenio_seq TO grp_atendimento;
GRANT USAGE, SELECT ON SEQUENCE consulta_id_consulta_seq TO grp_atendimento;
GRANT EXECUTE ON FUNCTION fn_consultas_a_realizar_por_data(DATE) TO grp_atendimento;
GRANT EXECUTE ON PROCEDURE sp_marcar_consulta(INT, INT, TIMESTAMP, TEXT) TO grp_atendimento;

-- Permissões do faturamento
GRANT USAGE ON SCHEMA public TO grp_faturamento;
GRANT SELECT ON paciente, medico, convenio, consulta, procedimento TO grp_faturamento;
GRANT SELECT, INSERT, UPDATE ON consulta_procedimento TO grp_faturamento;
GRANT SELECT, INSERT, UPDATE ON fatura, fatura_item TO grp_faturamento;
GRANT SELECT ON mv_faturamento_convenio_mensal TO grp_faturamento;
GRANT USAGE, SELECT ON SEQUENCE fatura_id_fatura_seq TO grp_faturamento;
GRANT USAGE, SELECT ON SEQUENCE fatura_item_id_fatura_item_seq TO grp_faturamento;
GRANT EXECUTE ON FUNCTION fn_consultas_realizadas_por_convenio(INT) TO grp_faturamento;
GRANT EXECUTE ON PROCEDURE sp_realizar_consulta_e_faturar(INT, JSONB) TO grp_faturamento;

-- Privilégios padrão para novos objetos criados pelo dono atual
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT ALL PRIVILEGES ON TABLES TO grp_admin_clinica;

ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT ALL PRIVILEGES ON SEQUENCES TO grp_admin_clinica;
```

INSTITUTO FEDERAL
Espírito Santo

```sql
-- Teste da [6] Política de segurança com grupos e usuários.
-- Simula um usuário da recepção para validar as permissões do grupo grp_atendimento.
SET ROLE usr_recepcao1;

SELECT current_user AS usuario_atual, current_role AS papel_atual;

-- Permitido: consulta de dados liberados ao grupo de atendimento.
SELECT * FROM medico ORDER BY id_medico LIMIT 3;
SELECT * FROM vw_agenda_consultas_dia ORDER BY data, horario LIMIT 3;

-- Permitido: uso da procedure de marcação de consulta.
CALL sp_marcar_consulta(
  1,
  1,
  '2026-12-30 11:00:00',
  'Teste de permissão de atendimento'
);
```

```sql
-- mostrar consulta marcada
SELECT * FROM consulta
WHERE id_paciente = 1 AND id_medico = 1 AND data_hora = '2026-12-30 11:00:00';

-- Deve falhar: usuário da recepção não pode inserir em fatura.
INSERT INTO fatura (id_convenio, data_emissao, valor_total, status)
VALUES (1, CURRENT_DATE, 0, 'PENDENTE');

RESET ROLE;
```

```
SET
 usuario_atual |  papel_atual
---------------+---------------
 usr_recepcao1 | usr_recepcao1
(1 row)

 id_medico |       nome       |   crm    | especialidade
-----------+------------------+----------+---------------
         1 | Dr. Pedro Santos | CRM12345 | Clínico Geral
         2 | Dra. Ana Costa   | CRM67890 | Cardiologia
(2 rows)

 id_consulta |    data    | horario |      medico      |    paciente    |  status
-------------+------------+---------+------------------+----------------+-----------
           1 | 2026-03-25 | 08:00   | Dr. Pedro Santos | João Silva     | REALIZADA
           2 | 2026-03-25 | 09:00   | Dra. Ana Costa   | Maria Oliveira | REALIZADA
           3 | 2026-03-25 | 10:00   | Dr. Pedro Santos | Carlos Souza   | AGENDADA
(3 rows)

CALL
 id_consulta | id_paciente | id_medico |      data_hora      |  status  |            observacao
-------------+-------------+-----------+---------------------+----------+-----------------------------------
           7 |           1 |         1 | 2026-12-30 11:00:00 | AGENDADA | Teste de permissão de atendimento
(1 row)

ERROR:  permission denied for table fatura
RESET
```

INSTITUTO FEDERAL
Espírito Santo

# [7] Escreva um stored procedure que utilize uma transação abrangendo:

```
1  BEGIN;
2    CALL sp_realizar_consulta_e_faturar(
3      2,
4      '[{"id_procedimento":1,"quantidade":1},{"id_procedimento":2,"quantidade":1}]'::jsonb
5    );
6  COMMIT;
```

```
COMMIT
```

INSTITUTO FEDERAL
Espírito Santo

```pg_procedure
CREATE OR REPLACE PROCEDURE sp_realizar_consulta_e_faturar(
  p_id_consulta INT,
  p_procedimentos JSONB
)
LANGUAGE plpgsql
AS $$
DECLARE
  v_status_consulta VARCHAR(20);
  v_id_paciente INT;
  v_data_consulta DATE;
  v_id_convenio INT;
  v_id_fatura INT;
  v_item JSONB;
  v_id_procedimento INT;
  v_quantidade INT;
  v_valor_unitario NUMERIC(10,2);
  v_valor_total NUMERIC(10,2) := 0;
  v_qtd_procedimentos INT;
BEGIN
  IF p_id_consulta IS NULL THEN
    RAISE EXCEPTION 'id_consulta é obrigatório.';
  END IF;

  SELECT c.status, c.id_paciente, c.data_hora::DATE
  INTO v_status_consulta, v_id_paciente, v_data_consulta
  FROM consulta c
  WHERE c.id_consulta = p_id_consulta
  FOR UPDATE;

  IF v_status_consulta IS NULL THEN
    RAISE EXCEPTION 'Consulta % não encontrada.', p_id_consulta;
  END IF;

  IF v_status_consulta = 'REALIZADA' THEN
    RAISE EXCEPTION 'Consulta % já está REALIZADA.', p_id_consulta;
  END IF;

  IF v_status_consulta = 'CANCELADA' THEN
    RAISE EXCEPTION 'Consulta % está CANCELADA e não pode ser faturada.', p_id_consulta;
  END IF;

  IF EXISTS (SELECT 1 FROM fatura_item WHERE id_consulta = p_id_consulta) THEN
    RAISE EXCEPTION 'Consulta % já possui item de fatura.', p_id_consulta;
  END IF;

  IF p_procedimentos IS NULL OR jsonb_typeof(p_procedimentos) <> 'array' THEN
    RAISE EXCEPTION 'Procedimentos devem ser informados em array JSONB.';
  END IF;

  v_qtd_procedimentos := jsonb_array_length(p_procedimentos);

  IF v_qtd_procedimentos = 0 THEN
    RAISE EXCEPTION 'Informe ao menos 1 procedimento.';
  END IF;

  IF v_qtd_procedimentos > 3 THEN
    RAISE EXCEPTION 'Uma consulta pode conter no máximo 3 procedimentos.';
  END IF;

  SELECT pc.id_convenio
  INTO v_id_convenio
  FROM paciente_convenio pc
  WHERE pc.id_paciente = v_id_paciente
    AND (pc.validade IS NULL OR pc.validade >= v_data_consulta)
  ORDER BY pc.validade DESC NULLS LAST
  LIMIT 1;
```

```pg_procedure
  IF v_id_convenio IS NULL THEN
    RAISE EXCEPTION 'Paciente da consulta % não possui convênio válido na data da consulta.', p_id_consulta;
  END IF;

  UPDATE consulta
  SET status = 'REALIZADA'
  WHERE id_consulta = p_id_consulta;

  FOR v_item IN SELECT * FROM jsonb_array_elements(p_procedimentos)
  LOOP
    v_id_procedimento := (v_item ->> 'id_procedimento')::INT;
    v_quantidade := COALESCE((v_item ->> 'quantidade')::INT, 1);

    IF v_id_procedimento IS NULL THEN
      RAISE EXCEPTION 'Cada item deve conter id_procedimento.';
    END IF;

    IF v_quantidade <= 0 THEN
      RAISE EXCEPTION 'Quantidade inválida para procedimento %.', v_id_procedimento;
    END IF;

    SELECT valor INTO v_valor_unitario
    FROM procedimento
    WHERE id_procedimento = v_id_procedimento;

    IF v_valor_unitario IS NULL THEN
      RAISE EXCEPTION 'Procedimento % não encontrado.', v_id_procedimento;
    END IF;

    INSERT INTO consulta_procedimento (id_consulta, id_procedimento, quantidade)
    VALUES (p_id_consulta, v_id_procedimento, v_quantidade)
    ON CONFLICT (id_consulta, id_procedimento)
    DO UPDATE SET quantidade = EXCLUDED.quantidade;

    v_valor_total := v_valor_total + (v_valor_unitario * v_quantidade);
  END LOOP;

  SELECT f.id_fatura
  INTO v_id_fatura
  FROM fatura f
  WHERE f.id_convenio = v_id_convenio
    AND f.status = 'PENDENTE'
    AND DATE_TRUNC('month', f.data_emissao) = DATE_TRUNC('month', CURRENT_DATE)
  ORDER BY f.id_fatura DESC
  LIMIT 1;

  IF v_id_fatura IS NULL THEN
    INSERT INTO fatura (id_convenio, data_emissao, valor_total, status)
    VALUES (v_id_convenio, CURRENT_DATE, 0, 'PENDENTE')
    RETURNING id_fatura INTO v_id_fatura;
  END IF;

  INSERT INTO fatura_item (id_fatura, id_consulta, valor)
  VALUES (v_id_fatura, p_id_consulta, v_valor_total);
END;
$$;
```

[8] Faça uma RULE que não permite a exclusão de uma consulta cujo STATUS é "REALIZADA".

```
1  DROP RULE IF EXISTS rl_impedir_delete_consulta_realizada ON consulta;
2  CREATE RULE rl_impedir_delete_consulta_realizada AS
3  ON DELETE TO consulta
4  WHERE OLD.status = 'REALIZADA'
5  DO INSTEAD NOTHING;
```

```
1  DELETE FROM consulta WHERE id_consulta = 1;
2  SELECT id_consulta, status FROM consulta WHERE id_consulta = 1;
```

```
DELETE 0
```

| | id_consulta [PK] integer | status character varying (20) |
|---|---|---|
| 1 | 1 | REALIZADA |

INSTITUTO FEDERAL
Espírito Santo

# [9] Escreva uma RULE que atualize o total da fatura sempre que um item da fatura for inserido.

```sql
DROP RULE IF EXISTS rl_atualizar_total_fatura ON fatura_item;
CREATE RULE rl_atualizar_total_fatura AS
ON INSERT TO fatura_item
DO ALSO
  UPDATE fatura
  SET valor_total = COALESCE((
    SELECT SUM(fi.valor)
    FROM fatura_item fi
    WHERE fi.id_fatura = NEW.id_fatura
  ), 0)
  WHERE id_fatura = NEW.id_fatura;
```

| | id_fatura [PK] integer | valor_total numeric (10,2) |
|---|---|---|
| 1 | 2 | 0.00 |

```
INSERT 0 1
```

| | id_fatura [PK] integer | valor_total numeric (10,2) |
|---|---|---|
| 1 | 2 | 100.00 |

```sql
INSERT INTO fatura_item (id_fatura, id_consulta, valor)
VALUES (2, 3, 100.00);

SELECT id_fatura, valor_total FROM fatura WHERE id_fatura = 2;
```

INSTITUTO FEDERAL
Espírito Santo